CBL RoboMaze Backlog

Advanced Topics: Path-finding algorithms, JavaFX

#	Name	How to Demo	Notes
1.	**Display the maze**	Launch the app; a grid-based maze is displayed in the GUI.	The grid should clearly distinguish walkable cells, walls and the target.
2.	**Display the robot**	Launch a level; the robot is displayed at its starting position.	Store the robot's position and direction in the Robot object.
3.	**Move the robot**	Start a level and use the movement controls; the robot moves to the corresponding adjacent cell.	Movement must update the robot's position.
4.	**Prevent invalid movement**	Move the robot towards a wall or outside the maze; the robot remains in place.	Prevent movement outside the grid and through obstacles.
5.	**Detect the target**	Move the robot onto the target cell; the application detects that the level has been completed.	Trigger the level-completion state.
6.	**Track moves**	Move the robot several times; the move counter is updated after each valid movement.	Invalid movements also increase the counter.
7.	**Reset a level**	Start a level, move the robot, and click Reset; the robot and relevant state return to their initial values.	Reset position, direction, move counter, and moves list.
8.	**Execute commands**	Enter a sequence of movement commands and click Run; the robot executes the commands sequentially.	Commands should be executed in the specified order.

9.	**Handle invalid commands**	Provide a command sequence that attempts to move through a wall; the application handles the invalid command without crashing.	Execution continues, but the robot doesn’t move
10.	**Find shortest path**	Place the robot and target in a maze and select Shortest Path; the smallest amount of tiles passed is calculated and displayed.	Use a BFS algorithm to find the shortest path through walkable cells, taking into account the number of turns.
11.	**Complete a level**	Reach the target; the application displays a completion message and allows the player to continue.	Store the completed-level state.
12.	**Display level statistics**	Complete a level; the application displays statistics such as moves used and shortest possible path.	Can be used to compare the player's solution with the optimal solution.
13.	**Progress through levels**	Complete a level and select New Maze; the next maze is loaded.	On button click, a dialog appears with options to set the number of rows and columns to the maze.
14.	**Level Generation**	Give options to the dialog; a new maze is automatically generated and displayed.	Generate the maze using a DFS algorithm. Check that the generated maze can be solved.